Seneca

SEG800
Digital Signal and Image Processing

Image Filtering
(In Spatial Domain)

Seneca College

[Slides adapted from references on the last slide]

Overview

- Noise
- Spatial Filtering
  - Separable filter kernels
- Spatial Sharpening

Noise [3]

- Noise: anything that degrades the ideal image
- Sources of noise:
  - The environment,
  - The imaging device,
  - Electrical interference,
  - The digitization process, and so on.

- Noise is additive and random:

$$\hat{I}(i,j) = I(i,j) + n(i,j)$$

Gaussian Noise [3]

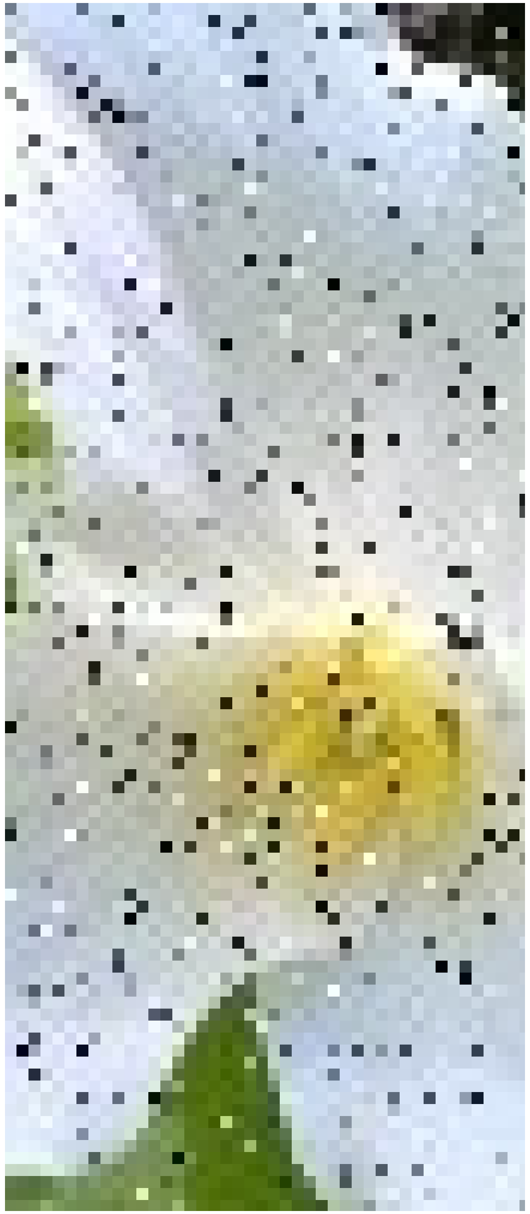
- A good approximation of real noise
- Modelled as a Gaussian (normal distribution with mean of 0)
- $n \sim N(\mu = 0, \sigma)$

Impulsive noise- Salt and Pepper Noise

- Impulsive: noise peaks or spikes
- Salt & Pepper is a model often used for impulsive noise
- Random values of brightness (darker or lighter)
- At random pixels the image

p = 0.1

p = 0.5



Correcting noise

- Observation: The image does not change sharply most of the time (low frequency), while noise is a sharp peak (high frequency)
- Therefore, using the values of the neighbors, we can often lower the noise
- Take the average of the neighboring pixels
  - This is equivalent to low-pass filtering
- Disadvantage: This will reduce the sharpness of edges in the image

Averaging

- The value at pixel (i, j) is calculated as the average of the pixels in its neighborhood
- Suitable for removing random noise, or smoothing

j →

i ↓

62	79	23	119	120	105	4	0
10	10	9	62	12	78	34	0
10	58	197	46	46	0	0	48
176	135	5	188	191	68	0	49
2	1	1	29	26	37	0	77
0	89	144	147	187	102	62	208
255	252	0	166	123	62	0	31
166	63	127	17	1	0	99	30

I_{in}

				?			

I_{out}

5x5 neighborhood

$$\text{new value} = \frac{9+62+\cdots+102+62}{25}$$

Spatial Filtering

Linear filtering

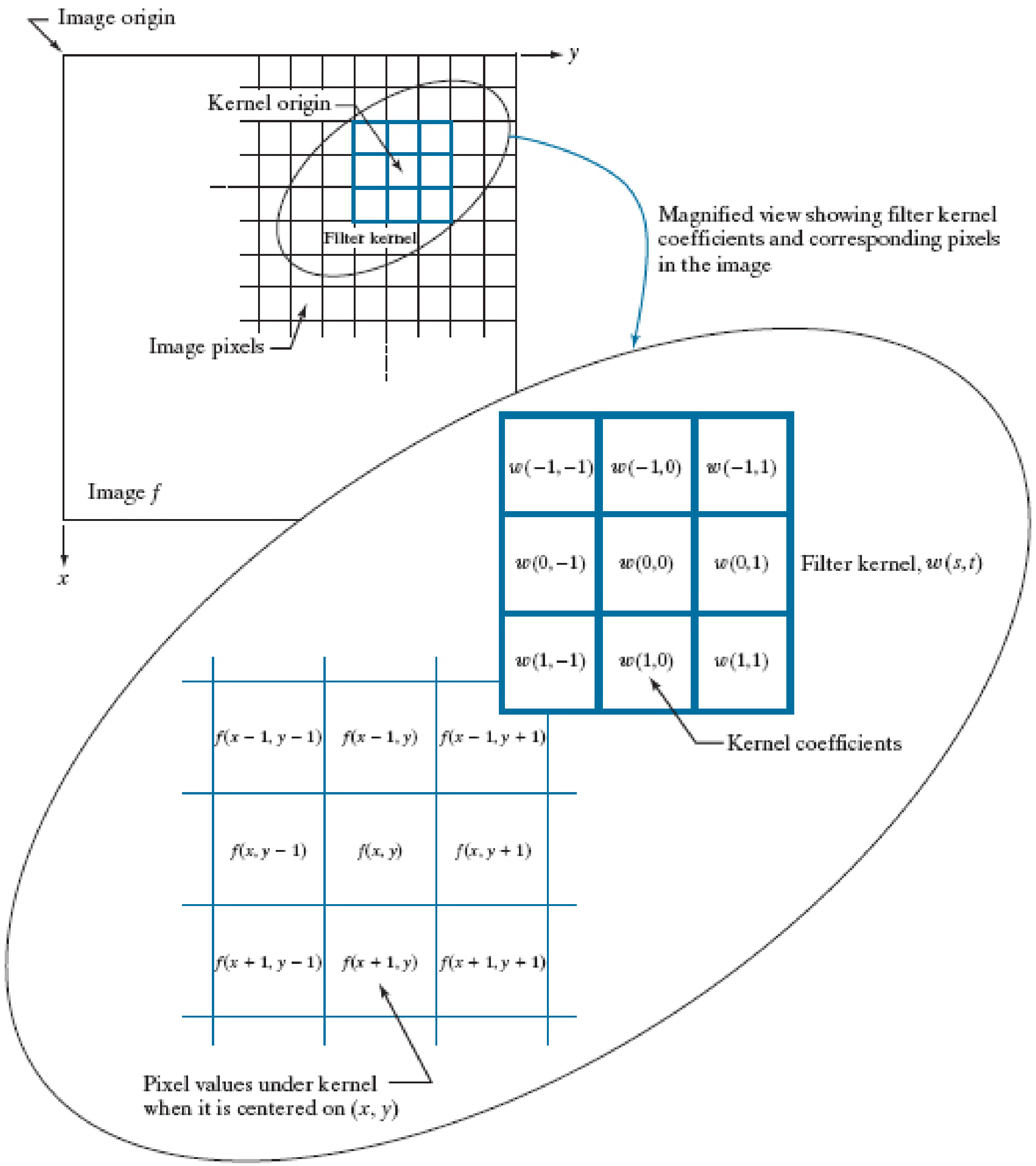
- **Filtering**: an algorithm that starts with some image f(x,y) and computes a new image g(x,y) using a neighborhood operator
- **Kernel (mask, template, window)**: defines the neighborhood and the operator (w)
- **Linear filter / linear kernel**: Values are calculated as a weighted sum of values in the neighborhood

$$g(x,y) = w(-1,-1)f(x-1,y-1) + w(-1,0)f(x-1,y) + \dots$$
$$+w(0,0)f(x,y) + \dots + w(1,1)f(x+1,y+1)$$

$$g(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t)f(x+s,y+t)$$

Figure 3.34

The mechanics of linear spatial filtering using a 3x3 kernel. The pixels are shown as squares to simplify the graphics. Note that the origin of the image is at the top left, but the origin of the kernel is at its center. Placing the origin at the center of spatially symmetric kernels simplifies writing expressions for linear filtering.



Pearson

Averaging- box kernel

- Averaging is equivalent to convolution with a box kernel

$$(w \star f)(x,y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

$j \rightarrow$, $i \downarrow$

62	79	23	119	120	105	4	0
10	10	9	62	12	78	34	0
10	58	197	46	46	0	0	48
176	135	5	188	191	68	0	49
2	1	1	29	26	37	0	77
0	89	144	147	187	102	62	208
255	252	0	166	123	62	0	31
166	63	127	17	1	0	99	30

$$k = 1/25 \times \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & \underline{\mathbf{1}} & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

5x5 (normalized) box kernel

$$k = 1/9 \times \begin{bmatrix} 1 & 1 & 1 \\ 1 & \underline{\mathbf{1}} & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

3x3 (normalized) box kernel

Examples

Salt & pepper noise with p = 0.1

After 5x5 box filter

Figure 3.39

(a) Test pattern of size 1024×1024 pixels. (b)-(d) Results of lowpass filtering with box kernels of sizes $3 \times 3, 11 \times 11,$ and 21×21, respectively.

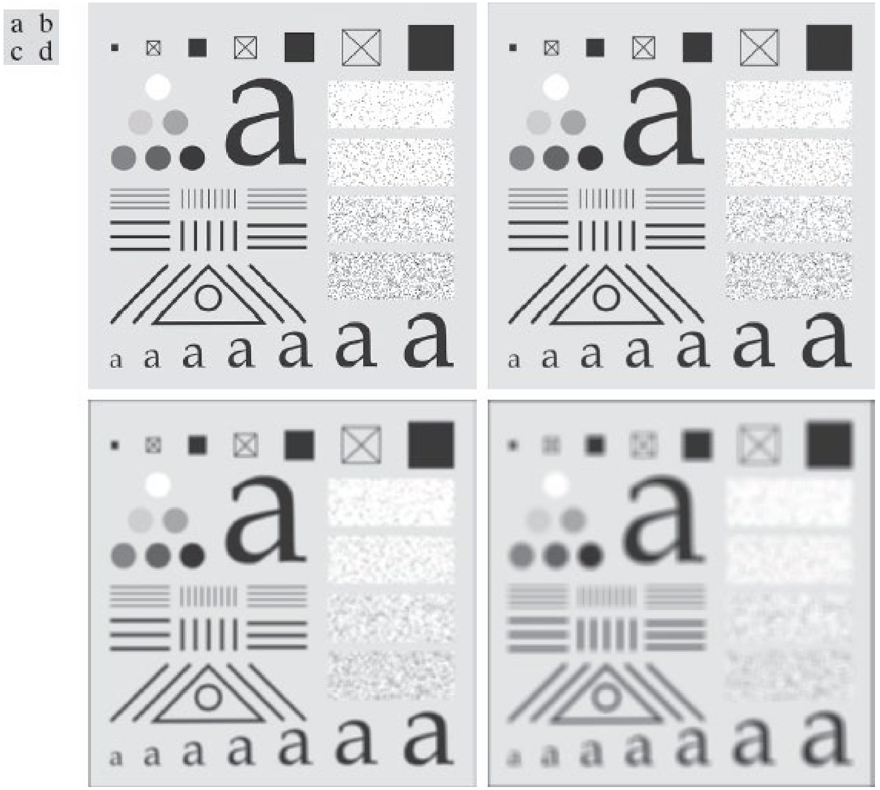
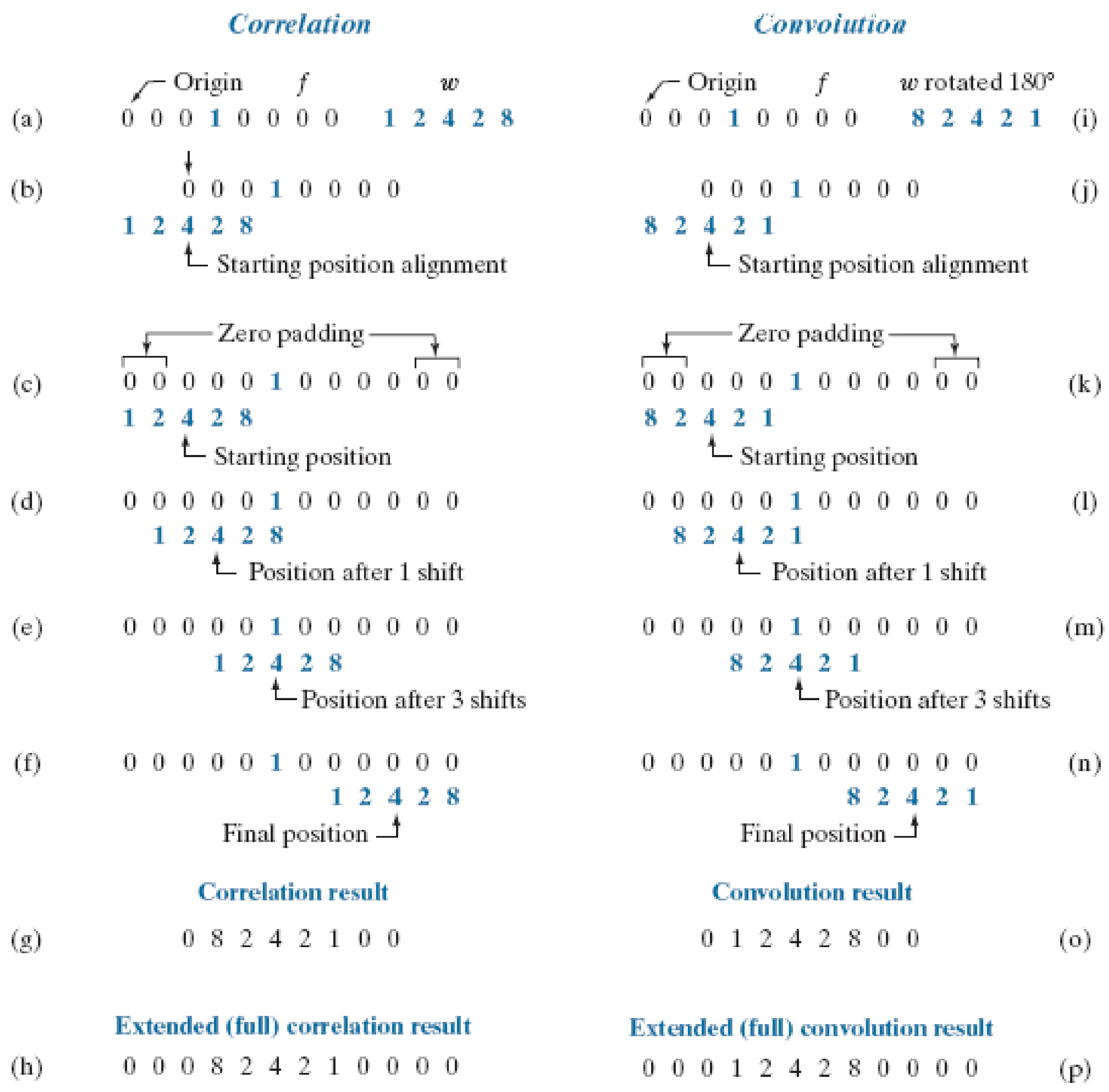


Figure 3.35

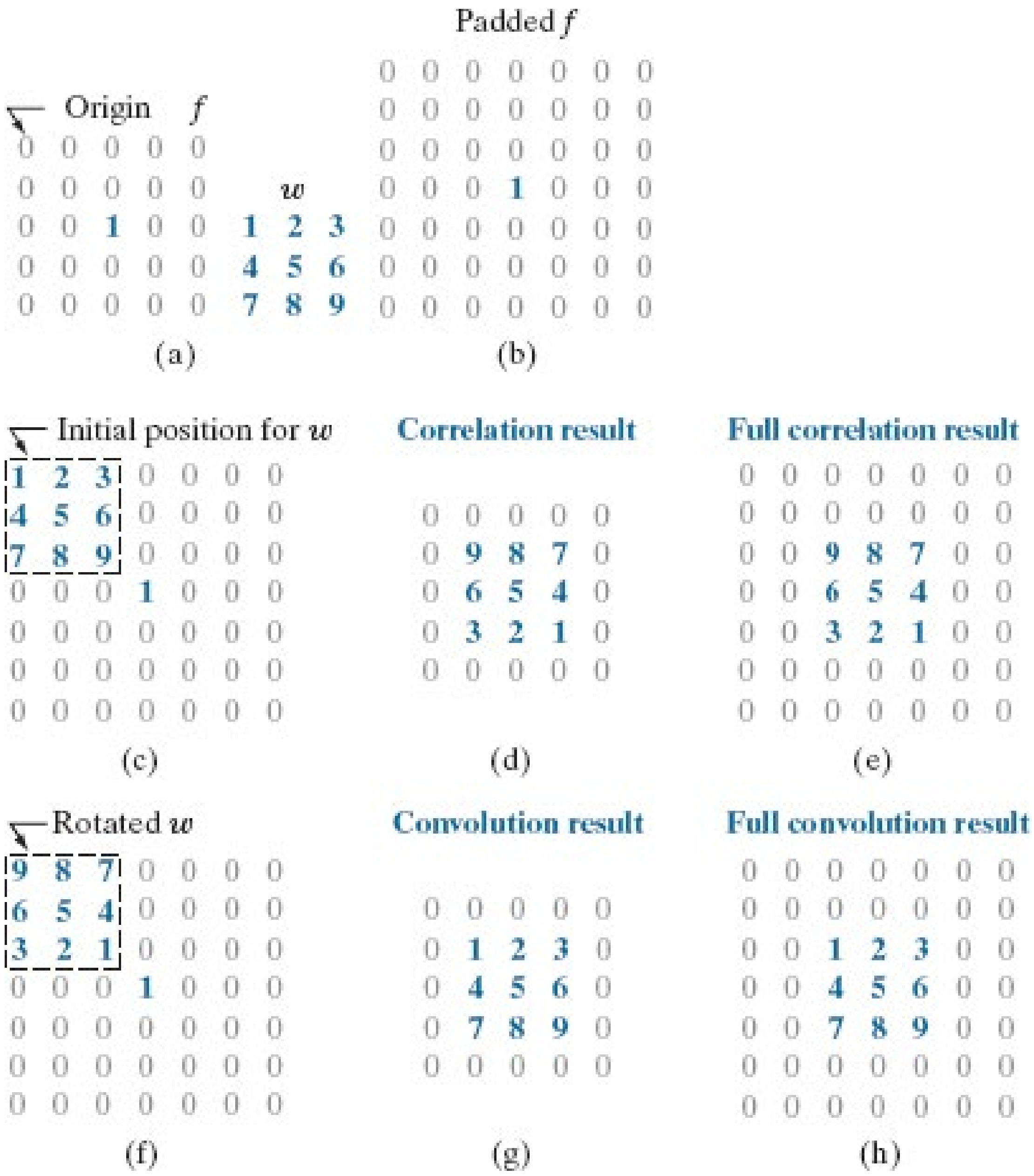
Illustration of 1-D correlation and convolution of a kernel, w, with a function f consisting of **a discrete unit impulse**. Note that correlation and convolution are functions of the variable x, which acts to **displace** one function with respect to the other. For the extended correlation and convolution results, the starting configuration places the rightmost element of the kernel to be coincident with the origin of f. Additional padding must be used.



Pearson

Figure 3.36

Correlation (middle row) and convolution (last row) of a 2-D kernel with an image consisting of a **discrete unit impulse**. The 0's are shown in gray to simplify visual analysis. Note that correlation and convolution are functions of x and y. As these variable change, they **displace** one function with respect to the other. See the discussion of Eqs. (3-45) and (3-46) regarding full correlation and convolution.



Pearson

Correlation and Convolution

- **Correlation** of a kernel w of size $m \times n$ with an image $f(x, y)$:

$$(w \star f)(x, y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x+s, y+t)$$

where $a = (m-1)/2$ and $b = (n-1)/2$

- **Convolution** of a kernel w of size $m \times n$ with an image $f(x, y)$:

$$(w \bigstar f)(x, y) = \sum_{s=-a}^{a} \sum_{t=-b}^{b} w(s,t) f(x-s, y-t)$$

Table 3.5

Some fundamental properties of convolution and correlation.
A dash means that the property does not hold.

Property	Convolution	Correlation
Commutative	$f \star g = g \star f$	—
Associative	$f \star (g \star h) = (f \star g) \star h$	—
Distributive	$f \star (g + h) = (f \star g) + (f \star h)$	$f \,☆\, (g + h) = (f \,☆\, g) + (f \,☆\, h)$

Separable filter kernels

Some filters are separable into smaller filters. Applying smaller filters is faster (faster implementation).

For example:
Convolving with

1/25 x

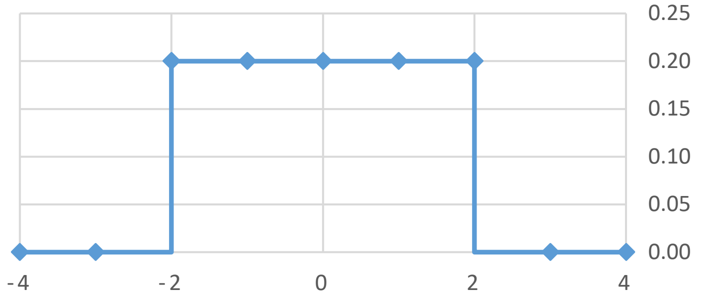
1	1	1	1	1
1	1	1	1	1
1	1	**1**	1	1
1	1	1	1	1
1	1	1	1	1

5x5 (normalized) box kernel

Is equivalent to
convolving with

1/5

1	1	1	1	1



And then
convolving with

1/5

1
1
1
1
1

Separable filter kernels (cont.)

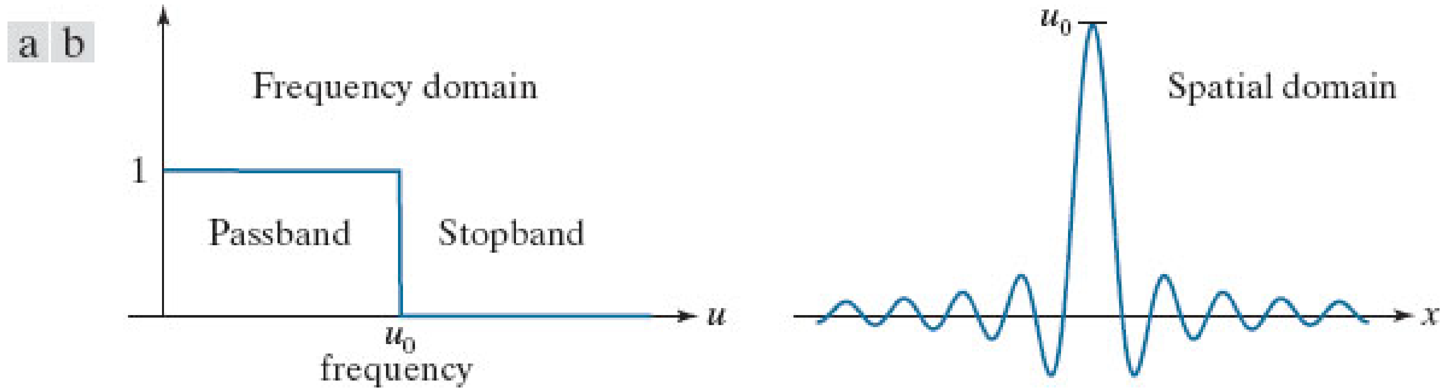
- For a separable kernel, w, that can be decomposed into two simpler kernels, w_1 and w_2, such that $w = w_1 \star w_2$, we have:

$$w \bigstar f = (w_1 \bigstar w_2) \bigstar f = (w_2 \bigstar w_1) \bigstar f = w_2 \bigstar (w_1 \bigstar f) = (w_1 \bigstar f) \bigstar w_2$$

- Image: M x N
- Kernel: m x n
- 2D convolution: MNmn multiplications & additions
- 1D convolution: MN(m+n) multiplications & additions

Figure 3.38

(a) Ideal 1-D lowpass filter transfer function in the frequency domain. (b) Corresponding filter kernel in the spatial domain.



Gaussian Filter (smoothing)

- The Gaussian Filter (2-D bell curve)

$$w(s,t) = G(s,t) = Ke^{-\frac{s^2+t^2}{2\sigma^2}} = Ke^{-\frac{r^2}{2\sigma^2}}$$

- Circularly symmetric (isotropic); response is independent of orientation
- Also, separable! It can be applied by first convolving with a 1D Gaussian Filter horizontally and then vertically

Salt & pepper noise with p = 0.1

After 5x5 box filter

Gaussian filter with sigma = 2.0

Gaussian filter with sigma = 1.0

Figure 3.40

Distances from the center for various sizes of square kernels.

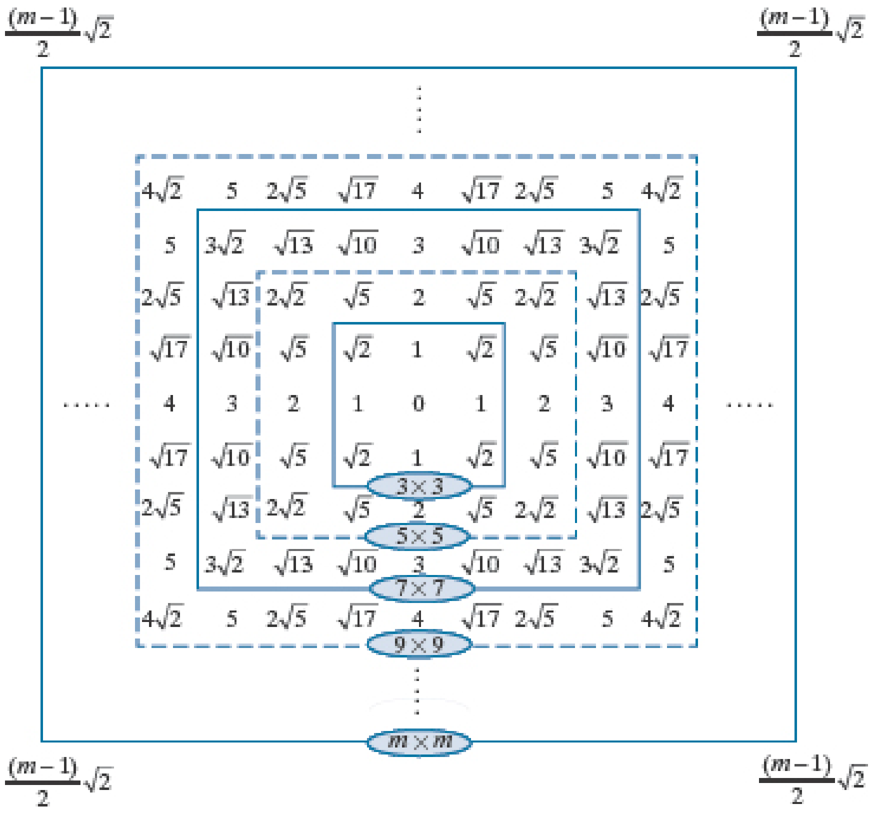
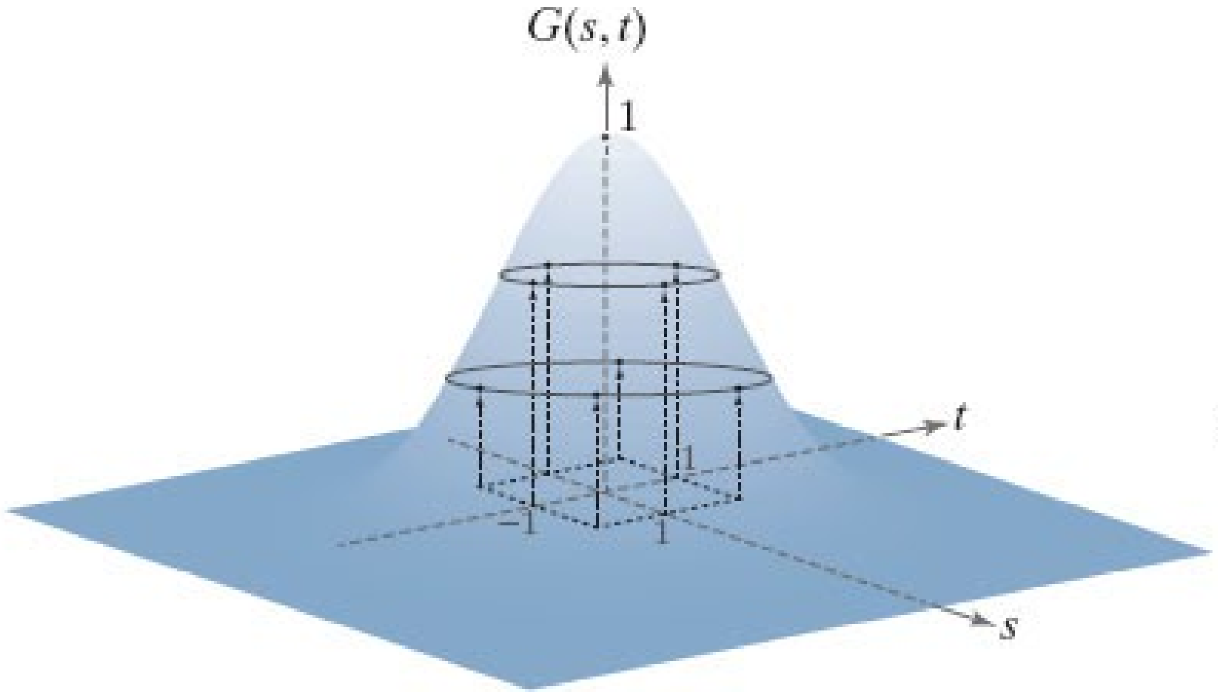


Figure 3.41

(a) Sampling a Gaussian function to obtain a discrete Gaussian kernel. The values shown are for $K=1$ and $\sigma=1$. (b) Resulting 3×3 kernel [this is the same as Fig. 3.37(b)].

a b



$$\frac{1}{4.8976}\times\begin{bmatrix}0.3679 & 0.6065 & 0.3679\\ 0.6065 & 1.0000 & 0.6065\\ 0.3679 & 0.6065 & 0.3679\end{bmatrix}$$

Table 3.6

Mean and standard deviation of the product (×) and convolution (★) of two 1-D Gaussian functions, f and g. These results generalize directly to the product and convolution of more than two 1-D Gaussian functions (see Problem 3.33).

	f	g	$f \times g$	$f \star g$
Mean	m_f	m_g	$m_{f \times g} = \dfrac{m_f \sigma_g^2 + m_g \sigma_f^2}{\sigma_f^2 + \sigma_g^2}$	$m_{f \star g} = m_f + m_g$
Standard deviation	σ_f	σ_g	$\sigma_{f \times g} = \sqrt{\dfrac{\sigma_f^2 \sigma_g^2}{\sigma_f^2 + \sigma_g^2}}$	$\sigma_{f \star g} = \sqrt{\sigma_f^2 + \sigma_g^2}$

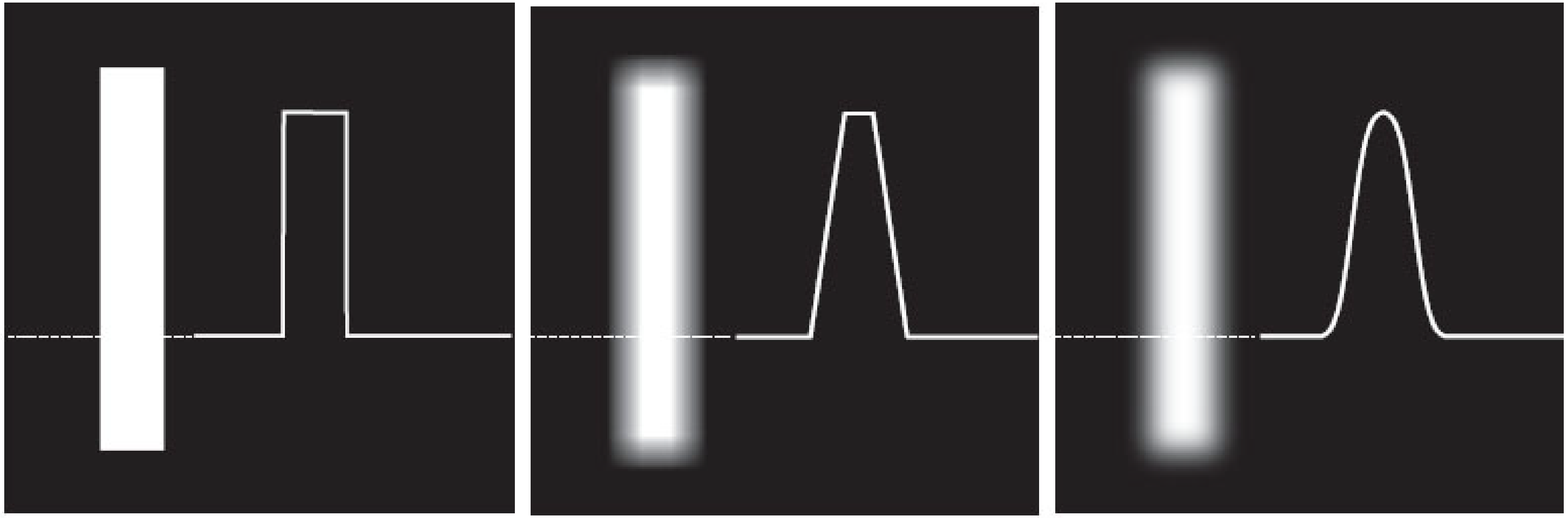
Figure 3.42

(a) A test pattern of size 1024×1024. (b) Result of lowpass filtering the pattern with a Gaussian kernel of size 21×21 with standard deviations $\sigma = 3.5$ (c) Result of using a kernel of size 43×43, with $\sigma = 7$. This result is comparable to Fig. 3.39(d). We used K = 1 in all cases.

a b c

Figure 3.44

(a) Image of a white rectangle on a black background, and a horizontal intensity profile along the scan line shown dotted. (b) Result of smoothing this image with a box kernel of size 71×71, and corresponding intensity profile. (c) Result of smoothing the image using a Gaussian kernel of size 151×151, with $K = 1$ and $\sigma = 25$. Note the smoothness of the profile in (c) compared to (b). The image and rectangle are of sizes 1024×1024 and 768×128 pixels, respectively



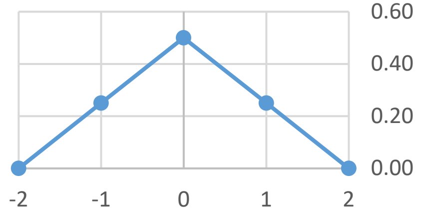
a b c

Bilinear kernel

- Also smoothing (removing noise)
- Equivalent to convolving with two separable 'tent' functions
- Example: 3x3 bilinear kernel:

1-D Tent kernel:

$$\frac{1}{4}\begin{array}{|c|c|c|}\hline 1 & 2 & 1 \\ \hline\end{array}$$



2-D Bilinear kernel:

$$\frac{1}{16}\begin{array}{|c|c|c|}\hline 1 & 2 & 1 \\ \hline 2 & 4 & 2 \\ \hline 1 & 2 & 1 \\ \hline\end{array}$$

Examples

p = 0.1

After 3x3 bilinear filter

Nonlinear filter

- The output pixel value is not a linear function of pixel values in the input
- Example: Median Filter
- The output value is the median of the pixels in the neighborhood

Examples

p = 0.1

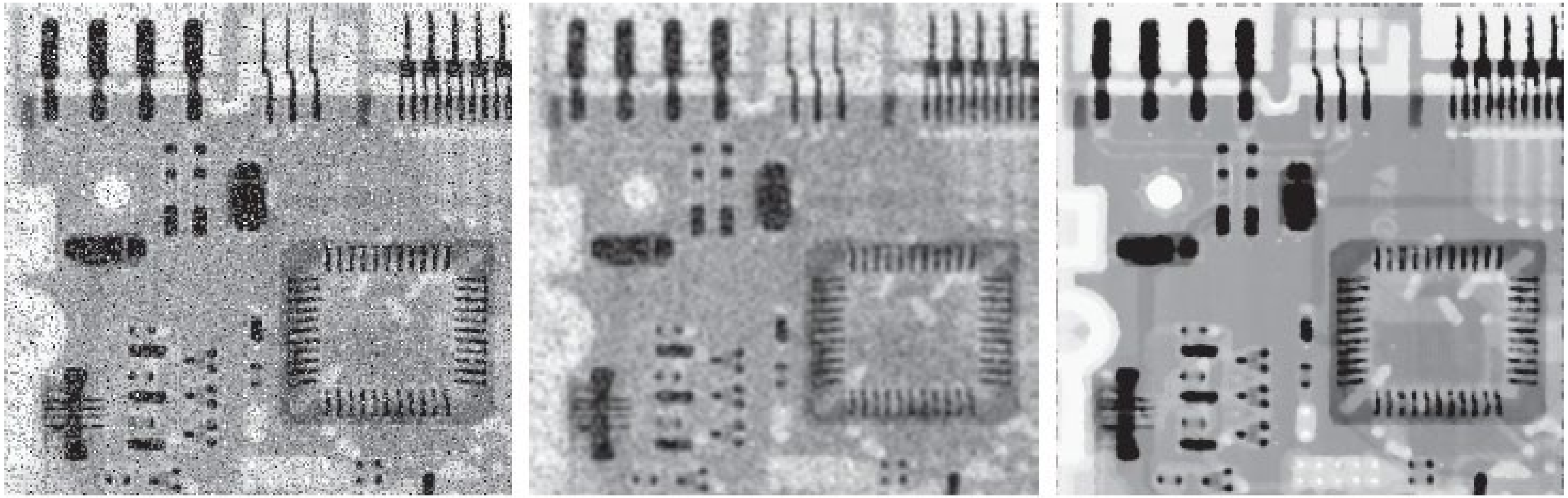
After a 5x5 median filter

Figure 3.49

(a) X-ray image of a circuit board, corrupted by salt-and-pepper noise. (b) Noise reduction using a 19×19 Gaussian lowpass filter kernel with $\sigma = 3$.

(c) Noise reduction using a 7×7 median filter.

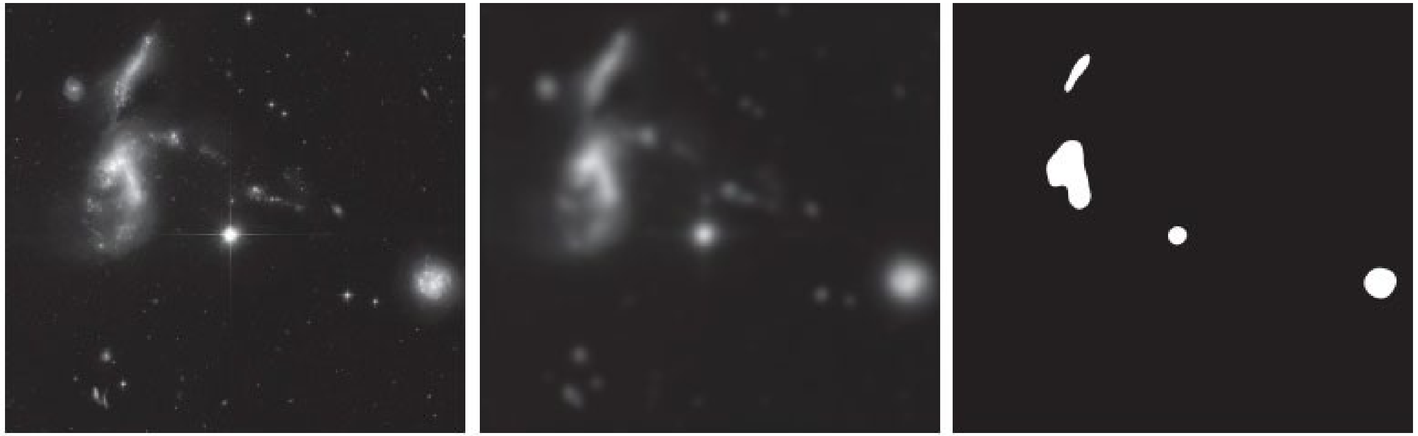
(Original image courtesy of Mr. Joseph E. Pascente, Lixi, Inc.)



a b c

Figure 3.47

(a) A 2566×2758, Hubble Telescope image of the Hickson Compact Group. (b) Result of lowpass filtering with a Gaussian kernel. (c) Result of thresholding the filtered image (intensities were scaled to the range [0, 1]). The Hickson Compact Group contains dwarf galaxies that have come together, setting off thousands of new star clusters. (Original image courtesy of NASA.)



a b c

Figure 3.48

(a) Image shaded by a shading pattern oriented in the −45 degree direction. (b) Estimate of the shading patterns obtained using lowpass filtering. (c) Result of dividing (a) by (b). (See Section 9.8 for a morphological approach to shading correction).

a b c

Spatial Sharpening

- Opposite to blurring/smoothing, highlights transitions in intensity.
- Opposite to averaging, can be accomplished by **differentiation**.
- Equivalent to highpass filtering.

- A first derivative definition requirements:
  - Must be zero in areas of constant intensity.
  - Must be nonzero at the onset of an intensity step or ramp.
  - Must be nonzero along intensity ramps.
- A second derivative definition requirements:
  - Must be zero in areas of constant intensity.
  - Must be nonzero at the onset and end of an intensity step or ramp.
  - Must be zero along intensity ramps

Derivatives

- For a one-dimensional function f(x):

$$\frac{\partial f}{\partial x} = f(x+1) - f(x)$$

$$\frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x)$$

- For a two-dimensional function f(x,y), the Laplacian is an isotropic derivative:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\nabla^2 f(x,y) = f(x+1,y) + f(x-1,y) + f(x,y+1) + f(x,y-1) - 4f(x,y)$$

Figure 3.50

(a) A section of a horizontal scan line from an image, showing ramp and step edges, as well as constant segments. (b) Values of the scan line and its derivatives. (c) Plot of the derivatives, showing a zero crossing. In (a) and (c) points were joined by dashed lines as a visual aid.

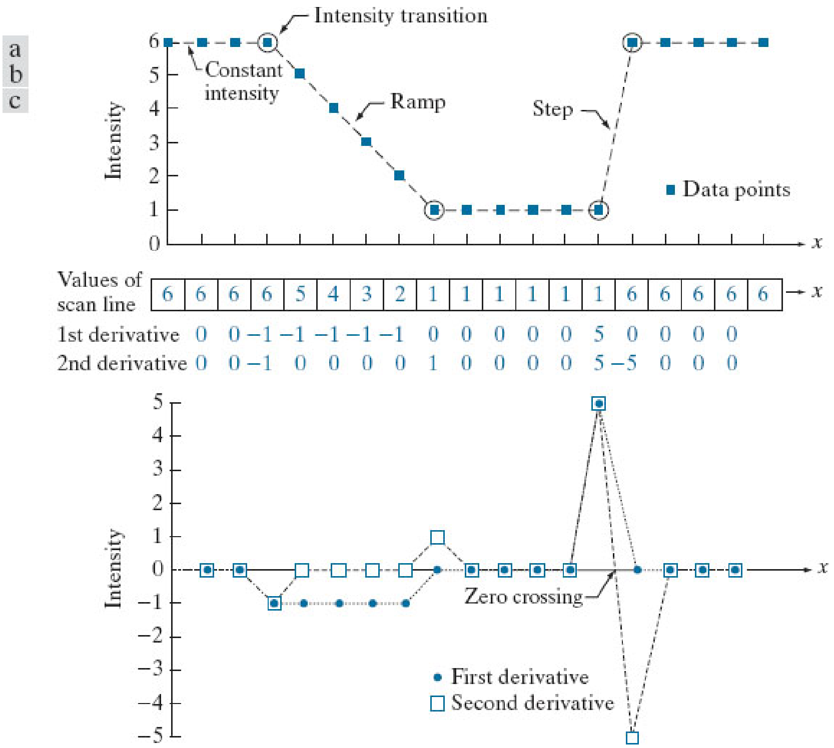


Figure 3.51

(a) Laplacian kernel used to implement Eq. (3-62). (b) Kernel used to implement an extension of this equation that includes the diagonal terms. (c) and (d) Two other Laplacian kernels.

0	1	0
1	−4	1
0	1	0

1	1	1
1	−8	1
1	1	1

0	−1	0
−1	4	−1
0	−1	0

−1	−1	−1
−1	8	−1
−1	−1	−1

a b c d

Pearson

Sharpening using the Laplacian

- Using the Laplacian, we can obtain a sharpened image by:

$$g(x, y) = f(x, y) + c\left[\nabla^2 f(x, y)\right]$$

- c= -1 (first two kernels on previous slide)
- c= 1 (last two kernels)
- c = sign (w(0,0))
- Note that sum of coefficients of kernel is zero, and therefore the sum of filtered image is zero. Some pixels have negative Laplacian values.

Figure 3.52

(a) Blurred image of the North Pole of the moon. (b) Laplacian image obtained using the kernel in Fig. 3.51(a). (c) Image sharpened using Eq. (3-63) with $c = -1$. (d) Image sharpened using the same procedure, but with the kernel in Fig. 3.51(b). (Original image courtesy of NASA.)

Pearson

Unsharp Masking

- A process of subtracting a smoothed version of the image from the original image to obtain a sharpened image.
- An old process used since 1930s in the printing industry

$$g_{\text{mask}}(x, y) = f(x, y) - \bar{f}(x, y)$$

$$g(x, y) = f(x, y) + k g_{\text{mask}}(x, y)$$

- k=1: **Unsharp masking**
- k<1: Reducing the effect
- k>1: **Highboost filtering**

Figure 3.54

1-D illustration of the mechanics of unsharp masking. (a) Original signal. (b) Blurred signal with original shown dashed for reference. (c) Unsharp mask. (d) Sharpened signal, obtained by adding (c) to (a).

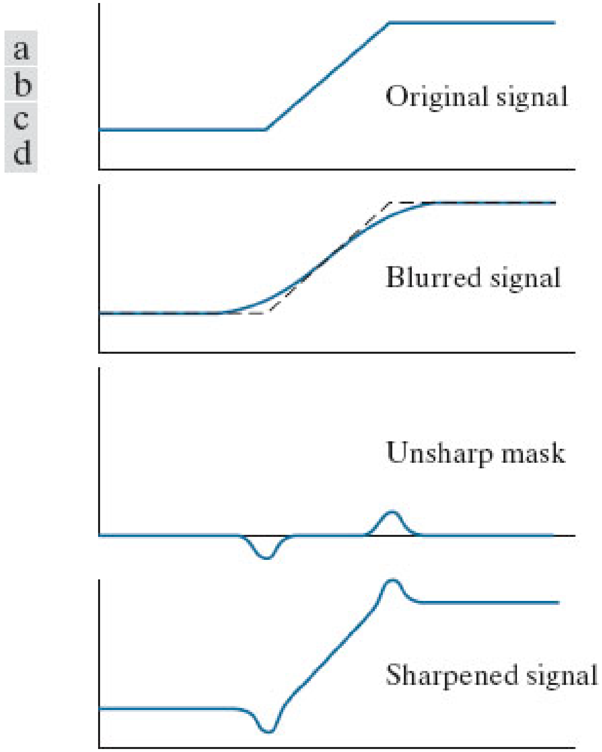


Figure 3.55

(a) Unretouched "soft-tone" digital image of size 469×600 pixels (b) Image blurred using a 31×31 Gaussian lowpass filter with $\sigma = 5$. (c) Mask. (d) Result of unsharp masking using Eq. (3-65) with $k = 1$. (e) and (f) Results of highboost filtering with $k = 2$ and $k = 3$, respectively.

Pearson

Sharpening using First Derivatives (Gradient)

- The gradient of an image at (x,y):

$$\nabla f \equiv \mathrm{grad}(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \dfrac{\partial f}{\partial x} \\ \dfrac{\partial f}{\partial y} \end{bmatrix}$$

- Magnitude of gradient vector:

$$M\left(x, y\right) = \|\nabla f\| = \mathrm{mag}\left(\nabla f\right) = \sqrt{g_x^2 + g_y^2}$$

$$M(x, y) \approx |g_x| + |g_y|$$

Figure 3.56

(a) A 3×3 region of an image, where the z_s are intensity values. (b)–(c) Roberts cross-gradient operators. (d)–(e) Sobel operators. All the kernel coefficients sum to zero, as expected of a derivative operator.

a
b c
d e

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

−1	0
0	1

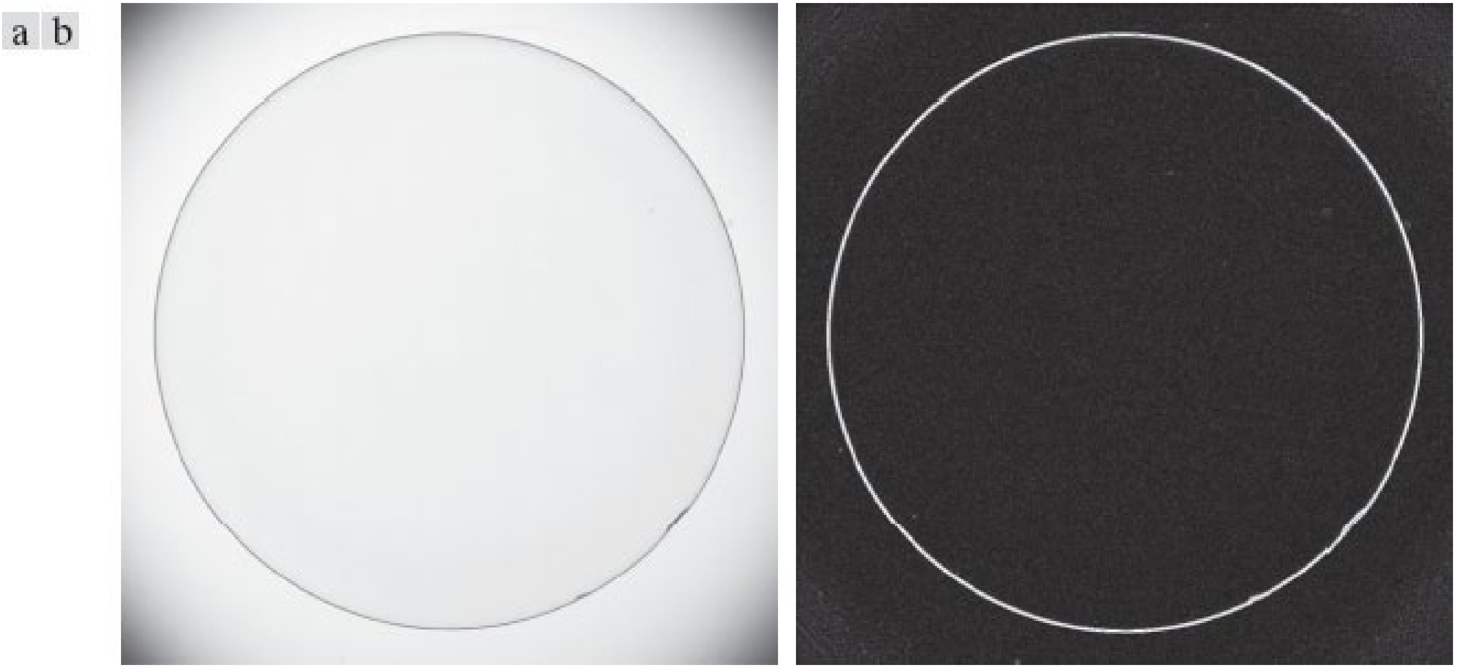
0	−1
1	0

−1	−2	−1
0	0	0
1	2	1

−1	0	1
−2	0	2
−1	0	1

Figure 3.57

(a) Image of a contact lens (note defects on the boundary at 4 and 5 o'clock). (b) Sobel gradient. (Original image courtesy of Perceptics Corporation.)



Summary

- Noise refers to anything that degrades the ideal image. Two mathematical models for noise are the Gaussian noise model and the Impulsive (or Salt & Pepper) noise model.
- Filtering is used for removing noise. With a linear filter, the output pixel value is a linear function of pixel values in the input(noisy) image. Common kernels are: box, Gaussian, and bilinear kernels. The median filter is a nonlinear filter that can remove noise, without blurring the image.
- Sharpening is the opposite of smoothing and can be done by using the first derivative, the Laplacian, or unsharp masking.

References

- [DIP] Gonzalez, R.C. and Woods, R.E., Digital Image Processing, Pearson, 4th edt., 2018.
- [CV] Computer Vision: Algorithms and Applications, R. Szeliski (http://szeliski.org/Book)
- [PICV] Practical introduction to Computer Vision with OpenCV, Kenneth Dawson-Howe, WILEY, 2014. Available through Seneca libraries

Copyright